$PM_{2.5}$ Forecasting in Kolkata Using Machine Learning Algorithms



By

Sreejani Roy

Department of Statistics

Bethune College

Supervisor

Dr. Saptarshi Mondal, Ph.D.

In partial fulfillment of the requirements for the degree of

Bachelor of Science in Statistics Honours

July, 2025

UNIVERSITY OF CALCUTTA

B.A./B.SC. SEMESTER – VI (HONS.)

EXAMINATION, 2025 (UNDER CBCS)

SUBJECT: STSA

PAPER: DSE B2 - Project Work

UNIVERSITY REGISTRATION NO.: 221-1211-0176-22

UNIVERSITY ROLL NO.: 223221-11-0010

INDEX

Contents **Page No**.

1. ACKNOWLEDGEMENT …… 2

2. EXECUTIVE SUMMARY …….. 3

3. INTRODUCTION …… 4-6

4. DATA ……. . 7

5. METHODOLOGY …… 8-17

6. RESULTS AND ANALYSIS …… 18-37

7. CONCLUSION ……. 38

8. REFERENCES …….. 39

APPENDIX …… 40-52

ACKNOWLEDGEMENT

I express my sincere gratitude to all who contributed to the completion of this project.

I am especially grateful to my supervisor, Dr. Saptarshi Mondal, for his invaluable guidance, expertise, and constant support. His insights greatly shaped the direction and quality of my work.

I also thank the faculty of the Department of Statistics, Bethune College, for providing a supportive academic environment.

My heartfelt thanks to my family and friends for their unwavering encouragement and belief in me, which sustained my dedication throughout.

Finally, I am thankful for the opportunity to undertake this project and for the knowledge and experience gained in the process.

EXECUTIVE SUMMARY

This project focuses on forecasting $PM_{2.5}$ concentrations in Kolkata by leveraging historical environmental data spanning from 2019 to 2024. $PM_{2.5}$ (particulate matter with a diameter of less than 2.5 micrometres) is widely recognized as a critical indicator of air quality, particularly in urban environments where vehicular emissions, industrial activities, and construction work contribute significantly to pollution levels.

Given the ability of $PM_{2.5}$ to penetrate deep into the respiratory tract, prolonged exposure poses serious health risks, especially in densely populated regions like Kolkata. Accurate short-term forecasting of $PM_{2.5}$ is essential for informing public health strategies, issuing timely air quality advisories, and implementing mitigation measures during periods of elevated pollution.

To address this, the project uses a dataset comprising 2192 rows of daily observations of $PM_{2.5}$, temperature, and humidity from 2019 to 2024, with meteorological data extended to January 2025. These environmental variables are known to influence air pollution dynamics and were included to enhance model performance.

Two ensemble learning techniques—Random Forest and XGBoost—were employed to model the complex, non-linear relationships among $PM_{2.5}$, temperature, and humidity. These models were trained using historical data and then used to generate daily forecasts for January 2025. Model performance was evaluated using RMSE and R^2, offering a comprehensive measure of accuracy and goodness of fit. The models achieved test RMSE values above 11 and below 13 with test R^2 scores close to 0.80, indicating strong predictive performance.

The findings of this project demonstrate how data-driven approaches can directly support proactive air quality management and timely public health interventions in urban environments.

INTRODUCTION

Air pollution is the single biggest environmental health risk .

- World Health Organisation (WHO)

Air pollution is a global challenge that transcends geographical boundaries, impacting health, visibility, and the environment. Among various pollutants, $PM_{2.5}$ — **particulate matter consisting of tiny solid particles and liquid droplets suspended in the air, including dust, soot, and smoke, with a diameter of less than 2.5 micrometers** — is considered especially dangerous due to its ability to penetrate deep into human lungs and enter the bloodstream. Rising $PM_{2.5}$ levels in urban areas, particularly in Kolkata, have become a significant concern, with the city frequently experiencing episodes of hazardous air quality, especially during winter months when temperature inversion traps pollutants closer to the ground.

$PM_{2.5}$ originates from diverse sources such as vehicle emissions, industrial discharges, biomass burning, and natural sources like dust storms. Exposure to elevated $PM_{2.5}$ levels is linked to respiratory problems, cardiovascular diseases, and premature death. **Effective monitoring and forecasting of $PM_{2.5}$ levels enable governments and citizens to take preventive actions during high-risk periods, reducing adverse health impacts.**

The Air Quality Index (AQI) is a standardized tool used to communicate how polluted the air currently, helping assess the health risks associated with different $PM_{2.5}$ concentrations:



This project analyses **daily $PM_{2.5}$ trends in Kolkata from 2019 to 2024, incorporating meteorological variables such as temperature and humidity (with data available up to January 31, 2025)**. The primary objective is to forecast $PM_{2.5}$ levels for January 2025 using machine learning algorithms, specifically **Random Forest and XGBoost**. These models are selected for their ability to capture complex, non-linear relationships within environmental data and their robustness to outliers and noise commonly found in pollution datasets. Forecast accuracy will be evaluated using metrics appropriate for regression tasks, including Root Mean Squared Error (RMSE) and R-squared (R^2), to provide a comprehensive assessment of model performance.

Temperature and humidity are the two most important weather factors affecting $PM_{2.5}$ pollution, especially in Kolkata. Temperature influences $PM_{2.5}$ by speeding up the chemical reactions that create tiny particles and by causing conditions like temperature inversions, where a layer of warm air traps cold air (and pollution) near the ground, preventing it from dispersing. Humidity makes these tiny particles absorb moisture and grow larger, which changes how long they stay in the air and how they behave chemically. This growth can increase pollution levels and affect visibility and health impacts. Since temperature and humidity have such a strong and consistent effect on $PM_{2.5}$ formation and movement, focusing on them alone provides enough information to accurately predict pollution levels in Kolkata. Other factors like wind or pressure have less impact here, so this study concentrates on temperature and humidity for effective forecasting.

January 2025 has been chosen for forecasting because this month historically experiences **higher pollution levels due to seasonal weather patterns, making accurate predictions especially important for timely public health measures and interventions**.

- India has experienced rising $PM_{2.5}$ pollution levels, particularly in major urban centers such as Delhi, Kolkata, and Mumbai.
- To combat this growing challenge, the Indian government has launched initiatives like the National Clean Air Programme (NCAP), where advanced predictive analytics play a crucial role in driving effective interventions.
- Reliable $PM_{2.5}$ forecasting models are essential for enabling timely health warnings, informed urban planning, and the formulation of more effective environmental policies.

- This study aims to deepen the understanding of air quality patterns and contribute to the development of data-driven strategies that promote a healthier and more sustainable future.

DATA

The dataset used in this project comprises daily observations recorded from January 1, 2019, to January 31, 2025. It includes four variables: Date (in DD/MM/YYYY format), $PM_{2.5}$ concentration, Temperature and Humidity.

The $PM_{2.5}$ data is available up to December 31, 2024, comprising a total of 2,192 records. The Temperature and Humidity data extend up to January 31, 2025, yielding 2,223 records for each variable.

A sample of the dataset is shown below:

DATE	PM2.5	TEMPERATURE	HUMIDITY
01-01-2019	209.31	16.9	67.4
02-01-2019	172.39	18.4	64.3
03-01-2019	244.09	17.8	66.2
04-01-2019	264.84	17.2	64.9
05-01-2019	219.26	17.2	66.2
06-01-2019	159.43	18.5	64.4
07-01-2019	158.89	19.1	66
08-01-2019	175.47	18.8	67.1
09-01-2019	189.25	18.4	68.5
10-01-2019	227.08	18	67.9
.	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.
21-01-2025		20.9	68.3
22-01-2025		21.3	70.5
23-01-2025		20.1	82.7
24-01-2025		20.8	80
25-01-2025		20.7	76
26-01-2025		20	60.7
27-01-2025		19.5	60.4
28-01-2025		20.5	66.2
29-01-2025		21.9	77.8
30-01-2025		23.9	81.4
31-01-2025		24.4	83.7

METHODOLOGY

Data Processing

The data processing in this project is carefully designed to prepare the dataset for forecasting of $PM_{2.5}$ concentrations in Kolkata using machine learning models. The raw dataset is read from an Excel file and contains daily measurements of $PM_{2.5}$, along with meteorological variables such as temperature and humidity. The date column is converted to proper Date format in R to ensure chronological ordering, which is essential for creating lag-based features and maintaining the temporal structure of the data.

A key part of the data preprocessing involves the creation of **lag variables**, which are previous-day values of $PM_{2.5}$. This is done to help the model learn temporal dependencies — for example, how the $PM_{2.5}$ levels today might depend on values from the last one, two, or three days. Specifically, three lag features (lag1_PM25, lag2_PM25, and lag3_PM25) are generated using the lag() function from the dplyr package. These lagged values serve as input features for the model, and any resulting rows with NA values (which occur at the beginning due to the lagging) are removed using na.omit() to ensure model compatibility.

To evaluate the model's performance, the dataset is split into training and testing subsets based on various proportions — 70%, 75%, 80%, and 85% to assess how model accuracy and generalization change with more training data and to determine the optimum model. The training set is used to fit the model, while the testing set is used to simulate future predictions and calculate errors like RMSE and R^2.

Once the initial model is trained using all lag and weather features, **feature importance** is extracted from the Random Forest and XGBoost model to determine which predictors contribute the most to $PM_{2.5}$ forecasting. The top 3 features are selected based on importance metrics like %IncMSE (% increase in Mean Squared Error) for Random Forest and Gain for XGBoost. A new model is then trained using only these top features to improve model efficiency and potentially reduce overfitting.

For forecasting $PM_{2.5}$ values for Jan 2025, machine learning algorithms : Random Forest and XGBoost has been used. These models were chosen over models like ARIMA, SARIMA, TBATS, Prophet, and hybrid approaches because they are better suited for handling the complexities of $PM_{2.5}$ data. Unlike traditional time series models that assume linearity, stationarity, or strong seasonality, these machine learning models can naturally capture non-

linear relationships and interactions between multiple variables such as lagged $PM_{2.5}$ values, temperature, and humidity. They are more robust to noise, missing data, and changing patterns, and they don't require extensive pre-processing or assumptions. Additionally, Random Forest and XGBoost provide feature importance metrics, allowing us to identify which factors influence $PM_{2.5}$ the most. Their adaptability, predictive power, and ability to model multivariate and complex environmental data confidently make them the ideal choice for accurate and interpretable forecasting in this project.

Random Forest

Mathematical Description and Application

In this study, a **Random Forest Regression model** is employed to forecast daily $PM_{2.5}$ levels in Kolkata using a combination of lagged $PM_{2.5}$ values, humidity, and temperature. Random Forest is an ensemble learning method that builds multiple decision trees using random subsets of the data and aggregates their predictions to improve accuracy and robustness.

Let Ln = {(Xi,Yi)}, i = 1(1) n be the training dataset of size n where n is the number of rows in the dataset and each Xi = (xi1,xi2,…,xip) is a feature vector consisting of :

- xi1,xi2,xi3: Lagged $PM_{2.5}$ values from the previous 1,2 and 3 days
- xi4: HUMIDITY
- xi5: TEMPERATURE

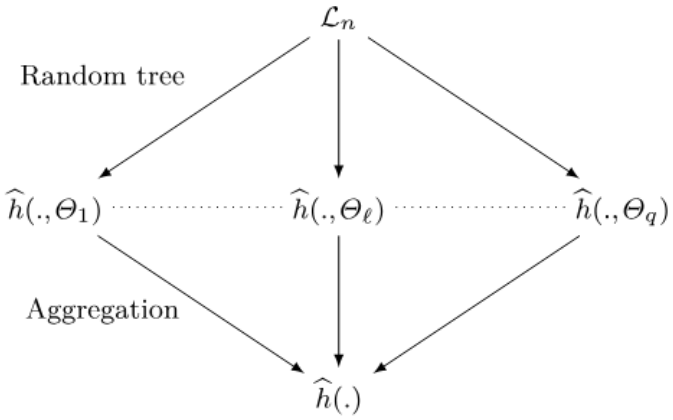
The response variable Yi is the observed $PM_{2.5}$ level on day i . The Random Forest model constructs q = 500 regression trees $\hat{h}(x,\Theta_1)$, $\hat{h}(x,\Theta_2)$,…., $\hat{h}(x,\Theta_q)$, each trained on a different random sample of the data, governed by a random variable Θ_ℓ. This randomization includes:

- **Bootstrapping**: Random sampling of the training data with replacement.
- **Feature subsampling**: At each node split, only a random subset of predictors is considered (controlled via the mtry parameter)

For a new input x, representing the conditions on a given day (i.e., the three lagged $PM_{2.5}$ values, humidity, and temperature), the final prediction is obtained by averaging the outputs of all individual trees using the formula :

$$\widehat{h}_{RF}(x) = \frac{1}{q}\sum_{\ell=1}^{q}\widehat{h}(x, \Theta_\ell)$$

The general scheme of Random Forest is thus given below:



The **aggregation process** used in the model reduces variance and increases predictive stability, making the model particularly well-suited for time-dependent environmental data.

Parameters Of Random Forest

ntree

DEFINITION : This parameter controls how many trees are built in the Random Forest ensemble. More trees usually improve model accuracy and reduce variance, but at a cost of increased computation time.

HOW IT HAS BEEN USED : ntree has been fixed at 500 across all the splits, which is a common default for Random Forest. It is large enough to ensure stable predictions but not too large to slow down training excessively.

WHY IS IT BENEFICIAL:

Averages over more trees implies more stable, lower variance predictions.

Helps avoid problems of unstable splits from decision trees.

Reliable and consistent across different configurations.

mtry

DEFINITION: mtry stands for **number of variables randomly sampled as candidates at each split** in a decision tree. In a Random Forest, each decision tree is built using a different random subset of the features. When making a decision point in a tree, Random Forest doesn't look at all the features. Instead, it randomly picks only **mtry** features and then finds the best way to divide the data using those.

HOW IT HAS BEEN USED: Standard mtry values = 1, 2, and 3 has been tested for every combination of nodesize and data split. By testing different values of mtry across various

data splits, we were able to observe how the number of features considered at each split influences model complexity, performance, and overfitting. This helped us understand the balance between tree diversity and predictive strength, and how selecting the right mtry value contributes to a more generalizable and stable model.

WHY IS IT BENEFICIAL:

Lower mtry increases randomness which leads to diverse trees implying reduced correlation between trees which in turn gives better generalization.

Higher mtry allows trees to be more similar and possibly more accurate on training data which increases the risk of overfitting.

The tuning used here helps balance these effects, discovering the best setting for the dataset.

Node size

DEFINITION:

This defines the smallest number of observations allowed in a terminal node (leaf). It controls **tree depth**: smaller nodesize means deeper trees; larger nodesize results in shallower trees.

HOW IT HAS BEEN USED: The program used for the project employed grid searching with standard node size values: 5, 10, 15, 20, alongside different values of mtry. Tuning nodesize helps balance model complexity and generalization. Smaller values allow deeper trees that capture more patterns but risk overfitting, while larger values make simpler, more robust trees. By testing multiple values, we find the best fit that ensures accuracy without instability.

WHY IS IT BENEFICIAL :

Small nodesize (e.g. 5) allows trees to capture fine patterns but may overfit.

Larger nodesize (e.g. 20) forces trees to be simpler and avoid capturing noise.

Provides a mechanism to **regularize the model** and control generalization capacity.

Importance Featuring

Feature importance in a Random Forest model tells us **how much each input variable contributes to the model's predictions**.

In simple terms, it measures **how useful or valuable** each feature is in helping the model make accurate predictions.

In Random Forest, feature importance is typically measured by observing how much the model's accuracy drops when the values of a feature are randomly shuffled. A **larger drop** means the model **relies heavily** on that feature, and so it's marked as more important.

This helps us **understand which variables are influencing the predictions the most and simplify the model by keeping only the most useful features**.

Thus, in this project, feature importance was calculated within a loop that tested multiple combinations of mtry and nodesize values and with ntree=500 for different train-test splits. For each combination, a full Random Forest model was first trained using all available features (lag1_PM25, lag2_PM25, lag3_PM25, HUMIDITY, and TEMPERATURE). From this model, feature importance scores (based on % increase in MSE) were extracted to identify the top 3 most influential variables. A second Random Forest model was then trained using only these top features, helping reduce complexity while maintaining accuracy. This process was repeated across different splits and hyperparameter combinations, allowing the model to be both well-tuned and interpretable by focusing on the most relevant predictors.

To select the best **Random Forest model** for forecasting $PM_{2.5}$, we systematically tested multiple configurations by varying mtry (the number of features considered at each split) and nodesize (the minimum number of observations in each terminal node), while keeping ntree = 500 fixed. These configurations were evaluated across multiple train-test split ratios to assess robustness. For each setting, a full Random Forest model was first trained using all available features (lagged $PM_{2.5}$, HUMIDITY, and TEMPERATURE). Feature importance was then computed using the **%IncMSE** metric, and the top three features were used to train a simpler, more focused model. This helped reduce complexity and mitigate overfitting while retaining the most informative predictors.

Model performance was evaluated using **RMSE** and **R^2** on both the training and test datasets, with emphasis on selecting models that performed well on unseen data. The final Random Forest model was chosen from among the **non-overfitting configurations**—those where the absolute difference between Train and Test RMSE was less than 2—by selecting the one with the **lowest combined RMSE** (Train RMSE + Test RMSE) which ensures we prioritize both accuracy and consistency across training and test performance—leading to a more reliable and robust forecast model.

Forecasting of $PM_{2.5}$ values for January 2025 was performed by retraining the final model. Care was taken to construct lag features using only previous observations (for example, lag 1 $PM_{2.5}$ value for January 1, 2025 was the $PM_{2.5}$ value of December 31,2024) to maintain temporal integrity. Predictions were then generated on the unseen portion of the dataset to

simulate a real-world forecasting scenario, and the results were evaluated through comparison with the actual $PM_{2.5}$ levels.

XGBoost

Mathematical Description and Application

XGBoost (Extreme Gradient Boosting) is an efficient implementation of gradient boosted decision trees. It trains a sequence of decision trees where each tree tries to fix the errors of the previous one, using both **first-order (gradient)** and **second-order (hessian)** information from the loss function.

We denote :

- x_i : input vector for the ith observation (e.g., lag1_PM25, lag2_PM25, lag3_PM25, TEMPERATURE, HUMIDITY
- y_i: target variable ($PM_{2.5}$ level)
- f(x) : the ensemble model we build step by step
- $f^{\wedge(m)}(x)$: the model at iteration m
- M: total number of boosting rounds,i.e, how many trees will be sequentially added to the model.
- α: learning rate which it controls how much each tree contributes to the final prediction.
- L(y,f(x)): loss function (squared error: $(y-f(x))^2$)

We first initialize the model:

We start with a constant prediction

$$\hat{f}_{(0)}(x) = \arg\min_{\theta} \sum_{i=1}^{N} L(y_i, \theta).$$

Where N is the size of the dataset after processing

For each boosting round, m=1 to M, we first compute gradients and hessians :

$$\hat{g}_m(x_i) = \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f(x)=\hat{f}_{(m-1)}(x)}.$$

$$\hat{h}_m(x_i) = \left[\frac{\partial^2 L(y_i, f(x_i))}{\partial f(x_i)^2}\right]_{f(x)=\hat{f}_{(m-1)}(x)}.$$

Then, we fit a decision tree or a base learner using the training set

$$\left\{ x_i, \frac{\hat{g}_m(x_i)}{\hat{h}_m(x_i)} \right\}_{i=1}^{N}$$

by solving the optimization problem below :

$$\hat{\phi}_m = \arg\min_{\phi \in \mathbf{\Phi}} \sum_{i=1}^{N} \frac{1}{2} \hat{h}_m(x_i) \left[\phi(x_i) - \frac{\hat{g}_m(x_i)}{\hat{h}_m(x_i)} \right]^2.$$

We then scale the learner as

$$\hat{f}_m(x) = \alpha \hat{\phi}_m(x).$$

Then we update the ensemble model as

$$\hat{f}_{(m)}(x) = \hat{f}_{(m-1)}(x) - \hat{f}_m(x).$$

We obtain the final prediction after M rounds using the formula :

$$\text{Output } \hat{f}(x) = \hat{f}_{(M)}(x) = \sum_{m=0}^{M} \hat{f}_m(x).$$

This is the sum of all scaled trees, giving the final $PM_{2.5}$ prediction.

Parameters of XG Boost

Eta

DEFINITION : α, the learning rate or eta controls how much influence each new decision tree has on the final model. It also scales the contribution of each tree before it is added to the model.

HOW IT IS USED : Standard values of eta = 0.01, 0.05, and 0.1 have been tested during the grid search for each data split and tree depth combination. This was done to identify the setting where the model is able to learn patterns in $PM_{2.5}$ variation without overfitting the training data while maintaining accuracy.

WHY IS IT BENEFICIAL :

Lower eta allows for smoother learning, preventing over-adjustment to noise in the data.

It helps the model generalize better, which is crucial for forecasting unseen data (e.g., January 2025).

Prevents the model from learning too quickly and overfitting.

max_depth

DEFINITION : max_depth sets the maximum depth of each decision tree in the XGBoost model. Deeper trees can capture more complex patterns but may also overfit the training data.

HOW IT HAS BEEN USED : Values 3, 5, and 7 have been evaluated for every data split and learning rate to determine the best depth that captures meaningful structure in the $PM_{2.5}$ data without fitting noise.

WHY IS IT BENEFICIAL**:**

Shallower trees (low max_depth) reduce model complexity and make it more generalizable.

Deeper trees allow more flexibility in modeling but may lead to overfitting.

The tuning ensures just enough complexity to capture temporal pollution trends without compromising forecast accuracy.

nrounds

DEFINITION : rounds refers to the number of boosting rounds — i.e., how many trees are sequentially added to the model. Each round attempts to correct errors from the previous model.

HOW IT HAS BEEN USED : A fixed value of nrounds = 100 has been strategically chosen to ensure the model has ample capacity to iteratively minimize error and learn complex patterns in the data, while maintaining computational efficiency. This strikes an ideal balance between predictive performance and training speed, and is well-suited for the pollution forecasting task at hand.

WHY IS IT BENEFICIAL :

A larger number of trees helps the model refine its predictions iteratively, reducing bias.

Works especially well when combined with small eta, allowing stable and gradual learning.

Prevents the model from stopping too early and missing important corrections.

Gain in XGBoost

In XGBoost, **Gain** measures how much a feature improves the model's performance each time it is used to split a node in the decision trees. It represents average reduction in the loss function brought by that feature across all trees.

In simpler words, the higher the gain, the more useful the feature is for accurate predictions.

In this project, Gain was used to determine **which features are the most important** for predicting $PM_{2.5}$ using XGBoost. For every combination of:

- eta (learning rate) = 0.01, 0.05, 0.1
- max_depth = 3, 5, 7
- train-test splits = 70%, 75%, 80%, 85%

XGBoost models were trained using all five features:
lag1_PM25, lag2_PM25, lag3_PM25, HUMIDITY, and TEMPERATURE and then the feature importance was extracted.

The **top 3 features with highest Gain** were selected for training a refined model.

Thus using Gain helped build compact and fast models by focusing only on high-impact features and avoiding noise or redundancy in the input variables. Gain provides direct insight into how useful each feature is in reducing prediction error. This makes the model more interpretable and allows it to be simplified by removing features with low Gain. As a result, the model becomes faster and more generalizable. By relying on only the most influential factors—such as lagged $PM_{2.5}$ values and humidity—the model **maintained high accuracy** while becoming **more efficient and robust across various hyperparameter settings and data splits.**

To select the best XGBoost model for forecasting $PM_{2.5}$, we systematically explored multiple configurations by varying the learning rate (eta) and maximum tree depth (max_depth) across different train-test split ratios, with nrounds = 100 fixed to allow sufficient model refinement while keeping training efficient. Each configuration involved training a full XGBoost model using all five predictors—lag1_PM25, lag2_PM25, lag3_PM25, HUMIDITY, and TEMPERATURE. Feature importance was then assessed using the Gain metric, and the top three features were selected to train a more focused and generalizable model.

Model performance was evaluated using RMSE and R^2 on both training and test sets. Only models satisfying the non-overfitting criterion—defined as an absolute difference between Train and Test RMSE less than 2—were considered. Among these, the final XGBoost model was chosen as the one having the lowest combined RMSE . In the context of $PM_{2.5}$ forecasting—where pollutant levels can fluctuate drastically due to weather, season, and human activity—a low combined RMSE indicates that the model captures the underlying trends accurately while keeping prediction errors minimal. This makes it dependable for real-world applications, such as issuing timely health advisories or informing policy decisions.

Forecasting $PM_{2.5}$ values for January 2025 was then performed using this final model, retrained on 85% of the data using the best-performing feature subset. Temporal consistency was

carefully maintained—lagged variables were created only from past observations (e.g., lag1_PM25 for January 1, 2025 was sourced from December 31, 2024). Forecasts for January were generated sequentially, using predicted values as lags where needed, to simulate a true out-of-sample forecasting environment. The results were then exported and plotted alongside actual $PM_{2.5}$ values to assess prediction accuracy visually.

Additionally, a combined comparison plot was generated, overlaying the predictions from both Random Forest and XGBoost models against the actual $PM_{2.5}$ values. This approach provides a clear and informative visual assessment of each model's performance. By presenting all three curves together, it becomes easier to accurately evaluate how closely each model tracks real trends, detect any lag or deviation, and understand their relative strengths. This type of visualization is both insightful and essential—it supports effective model interpretation, validates forecasting reliability, and ensures informed decision-making based on model behaviour in a real-world context.

RESULTS AND ANALYSIS

Random Forest

To ensure a robust model selection process, a comprehensive grid search was conducted across various combinations of hyperparameters—namely, mtry (1 to 3), nodesize (5, 10, 15, 20), and train-test split ratios (70%, 75%, 80%, and 85%) with ntree fixed at 500. For each configuration, a Random Forest model was first trained using all available features. The model's feature importances were then evaluated using the %IncMSE metric, which quantifies how much predictive error increases when each feature is permuted. From this, the top three most important predictors were extracted and used to retrain a focused model. After training the reduced model, performance was assessed using both RMSE and R^2 metrics on the training and testing datasets. A model was deemed overfitted if the train and test RMSE differed by more than or equal to 2 units.

An overview of all the combinations along with their top 3 important features, Train and Test RMSE, Train and Test R^2 values and their overfitting status is given below:

	Split_Percent	mtry	nodesize	Features	Train_RMSE	Test_RMSE	Train_R2	Test_R2	Overfitting
1	70	1	5	HUMIDITY, lag1_PM25, TEMPERATURE	9.699	14.530	0.969	0.805	Yes
2	70	1	10	lag1_PM25, HUMIDITY, TEMPERATURE	11.878	14.619	0.952	0.806	Yes
3	70	1	15	HUMIDITY, lag1_PM25, TEMPERATURE	13.192	14.952	0.941	0.803	No
4	70	1	20	lag1_PM25, HUMIDITY, TEMPERATURE	14.070	14.972	0.933	0.804	No
5	70	2	5	lag1_PM25, HUMIDITY, TEMPERATURE	8.925	13.323	0.973	0.810	Yes
6	70	2	10	lag1_PM25, HUMIDITY, TEMPERATURE	10.981	13.205	0.958	0.815	Yes
7	70	2	15	lag1_PM25, HUMIDITY, TEMPERATURE	12.282	13.152	0.948	0.814	No
8	70	2	20	lag1_PM25, HUMIDITY, TEMPERATURE	13.258	12.989	0.939	0.818	No
9	70	3	5	lag1_PM25, HUMIDITY, TEMPERATURE	8.757	13.387	0.974	0.809	Yes
10	70	3	10	lag1_PM25, HUMIDITY, TEMPERATURE	10.684	13.267	0.960	0.811	Yes
11	70	3	15	lag1_PM25, HUMIDITY, TEMPERATURE	12.017	13.159	0.950	0.812	No
12	70	3	20	lag1_PM25, HUMIDITY, TEMPERATURE	12.995	13.099	0.941	0.813	No
13	75	1	5	lag1_PM25, HUMIDITY, TEMPERATURE	9.522	14.210	0.969	0.827	Yes
14	75	1	10	lag1_PM25, HUMIDITY, TEMPERATURE	11.666	14.419	0.953	0.828	Yes
15	75	1	15	HUMIDITY, lag1_PM25, TEMPERATURE	12.955	14.608	0.941	0.828	No
16	75	1	20	HUMIDITY, lag1_PM25, TEMPERATURE	13.849	14.667	0.932	0.827	No
17	75	2	5	lag1_PM25, HUMIDITY, TEMPERATURE	8.752	13.077	0.973	0.835	Yes
18	75	2	10	lag1_PM25, HUMIDITY, TEMPERATURE	10.797	12.999	0.958	0.836	Yes
19	75	2	15	lag1_PM25, HUMIDITY, TEMPERATURE	12.069	12.893	0.948	0.839	No
20	75	2	20	lag1_PM25, HUMIDITY, TEMPERATURE	13.027	12.785	0.939	0.841	No
21	75	3	5	lag1_PM25, HUMIDITY, TEMPERATURE	8.585	13.209	0.974	0.831	Yes
22	75	3	10	lag1_PM25, HUMIDITY, TEMPERATURE	10.557	13.098	0.960	0.835	Yes
23	75	3	15	lag1_PM25, HUMIDITY, TEMPERATURE	11.838	13.009	0.950	0.836	No
24	75	3	20	lag1_PM25, HUMIDITY, TEMPERATURE	12.782	12.938	0.941	0.835	No
25	80	1	5	lag1_PM25, HUMIDITY, TEMPERATURE	9.240	15.147	0.970	0.825	Yes
26	80	1	10	HUMIDITY, lag1_PM25, TEMPERATURE	11.390	15.253	0.954	0.827	Yes
27	80	1	15	HUMIDITY, lag1_PM25, TEMPERATURE	12.602	15.383	0.943	0.825	Yes
28	80	1	20	HUMIDITY, lag1_PM25, TEMPERATURE	13.458	15.629	0.935	0.824	Yes
29	80	2	5	lag1_PM25, HUMIDITY, TEMPERATURE	8.559	13.744	0.974	0.833	Yes
30	80	2	10	lag1_PM25, HUMIDITY, TEMPERATURE	10.526	13.626	0.960	0.834	Yes
31	80	2	15	lag1_PM25, HUMIDITY, TEMPERATURE	11.788	13.560	0.949	0.835	No
32	80	2	20	lag1_PM25, HUMIDITY, TEMPERATURE	12.673	13.465	0.941	0.837	No
33	80	3	5	lag1_PM25, HUMIDITY, TEMPERATURE	8.358	14.012	0.975	0.827	Yes
34	80	3	10	lag1_PM25, HUMIDITY, TEMPERATURE	10.230	13.778	0.962	0.830	Yes
35	80	3	15	lag1_PM25, HUMIDITY, TEMPERATURE	11.572	13.689	0.951	0.831	Yes
36	80	3	20	lag1_PM25, HUMIDITY, TEMPERATURE	12.476	13.648	0.943	0.832	No
37	85	1	5	HUMIDITY, lag1_PM25, TEMPERATURE	9.206	13.457	0.969	0.791	Yes
38	85	1	10	HUMIDITY, lag1_PM25, TEMPERATURE	11.222	13.546	0.953	0.790	Yes
39	85	1	15	HUMIDITY, lag1_PM25, TEMPERATURE	12.545	13.759	0.942	0.790	No
40	85	1	20	lag1_PM25, HUMIDITY, TEMPERATURE	13.397	13.852	0.933	0.791	No
41	85	2	5	lag1_PM25, HUMIDITY, TEMPERATURE	8.472	12.174	0.973	0.804	Yes
42	85	2	10	lag1_PM25, HUMIDITY, TEMPERATURE	10.386	12.196	0.959	0.803	No
43	85	2	15	lag1_PM25, HUMIDITY, TEMPERATURE	11.663	12.065	0.948	0.803	No
44	85	2	20	lag1_PM25, HUMIDITY, TEMPERATURE	12.583	12.065	0.939	0.805	No
45	85	3	5	lag1_PM25, HUMIDITY, TEMPERATURE	8.244	12.210	0.975	0.801	Yes
46	85	3	10	lag1_PM25, HUMIDITY, TEMPERATURE	10.121	12.253	0.961	0.800	Yes
47	85	3	15	lag1_PM25, HUMIDITY, TEMPERATURE	11.421	12.194	0.950	0.801	No
48	85	3	20	lag1_PM25, HUMIDITY, TEMPERATURE	12.366	12.135	0.941	0.802	No

From all the non-overfitting configurations, the model with the **lowest total RMSE (which ensures the model performs well on both training and test data, indicating strong generalization and accuracy)** was selected as the best-performing configuration.

	Split_Percent	mtry	nodesize	Features	Train_RMSE	Test_RMSE	Train_R2	Test_R2	Overfitting	Total_RMSE
1	85	2	10	lag1 PM25, HUMIDITY, TEMPERATURE	10.386	12.196	0.959	0.803	No	22.582

The selected model—trained with 85% of the data, mtry = 2, and nodesize = 10—emerged as the best-performing configuration based on both statistical evidence and sound machine learning principles. The 85% training split ensures the model has access to a large, representative dataset to learn complex patterns, while the remaining 15% is reserved for a robust evaluation. The mtry = 2 value strikes an ideal balance by incorporating randomness to reduce correlation among trees, while still ensuring key predictors are regularly considered at each decision point. A nodesize = 10 further prevents overfitting by ensuring each terminal node contains a reasonable number of observations, smoothing predictions and enhancing generalizability. Setting ntree = 500 in Random Forest provides a strong balance between performance and computational efficiency. With 500 trees, the model benefits from the law of large numbers—averaging predictions over a large number of trees reduces variance and helps stabilize predictions. This results in more reliable and consistent outputs, especially when working with noisy or complex data. Additionally, with more trees, there's a greater chance that all important features are adequately considered across different splits, improving the model's ability to generalize. The larger ensemble also enhances the accuracy of feature importance scores, offering better interpretability.

Achieving a **Test RMSE below 13** is particularly meaningful, as it signals **high-accuracy forecasting** even in the face of real-world variability in air quality. $PM_{2.5}$ levels in urban environments can vary dramatically due to seasonal patterns, weather changes, and human activity. A model with this level of test accuracy demonstrates strong generalization ability—it learns real patterns rather than memorizing noise.

Furthermore, maintaining a **combined RMSE under 23** ensures that the model is not only effective on new data but also consistent during training. This balance is essential: a model with very low training error but high test error likely overfits, while one with both moderately low and closely matched RMSEs is more stable and reliable.

Since in this project, we are forecasting daily $PM_{2.5}$ levels over several years for a major city like Kolkata, where values typically fluctuate widely due to seasonal and environmental variability. Considering the observed $PM_{2.5}$ values in the dataset often range between 3 $\mu g/m^3$

and 300 µg/m^3, a RMSE under 13 means that the typical prediction error is quite small compared to the actual pollution levels. Similarly, a **combined RMSE close to 23** (Train RMSE + Test RMSE) shows that the model is maintaining good performance across both known and unseen data, suggesting consistency and generalization. This level of accuracy is meaningful in an environmental context where even small changes in $PM_{2.5}$ concentration can significantly affect air quality categories and health advisories.

While feature importance was computed for every model, displaying the importance output for all models would be redundant, since the rankings across different configurations were largely similar and did not add further insight. To maintain clarity in interpretation and avoid repetition, we present only the importance ranking from the best-performing configuration.

```
Split: 85 % | mtry: 2 | nodesize: 10
      Feature Importance
1   lag1_PM25   55.69500
2    HUMIDITY   43.55885
3 TEMPERATURE   28.79281
4   lag3_PM25   22.90568
5   lag2_PM25   20.96859
```

It has been observed that across the different Random Forest configurations the features lag1_PM25, TEMPERATURE, and HUMIDITY consistently emerged as the most important predictors of current $PM_{2.5}$ levels. This is both expected and justified. The high importance of **lag1_PM25** reflects the strong **temporal autocorrelation** in air pollution data - $PM_{2.5}$ levels on a given day are highly influenced by those of the preceding day, making this lag a powerful and reliable predictor. In the best-performing model, its importance score is significantly higher than that of any other variable, representing its dominant role. Environmental variables like **TEMPERATURE** and **HUMIDITY** also ranked highly. Their influence on $PM_{2.5}$ levels is widely acknowledged: high humidity helps keep particles in the air for longer periods by slowing their settling, while temperature affects how stable the air is and influences the chemical changes that lead to the formation and spread of particulate matter**.** Their consistently strong importance scores across models confirm that the Random Forest not only captures statistical patterns but also reflects meaningful physical relationships. In comparison, **lag2_PM25** and **lag3_PM25** showed much lower importance scores (around 21–23), indicating diminishing predictive value as the lag increases. This suggests that while short-term memory (lag1) is critical, additional lags add limited new information.

To evaluate if forecast accuracy could be further enhanced, a model using all 5 features was also evaluated using the same process of grid search. The best-performing model is given below along with the Train and Test RMSE. Train and Test R^2 values:

	Split_Percent	mtry	nodesize	Features	Train_RMSE	Test_RMSE	Train_R2	Test_R2	Overfitting	Total_RMSE
1	85	2	10	lag1_PM25, lag2_PM25, lag3_PM25, HUMIDITY, TEMPERATURE	9.978	11.801	0.963	0.807	No	21.779

While this model demonstrated slightly lower Train and Test RMSE and higher R^2 values compared to the three-features model, the differences were marginal.

The forecasts were generated using the hyperparameters of the best-performing model. The values of lag1_PM25, lag2_PM25, and lag3_PM25 were set using the $PM_{2.5}$ observations from December 31, December 30, and December 29 of 2024, respectively. The corresponding HUMIDITY and TEMPERATURE inputs were taken from the actual recorded values for each day of January 2025. The forecasts for January 2025 are given below:

DATE	HUMIDITY	TEMPERATURE	Predicted_PM25
2025-01-01	72.7	18.5	79.69133756
2025-01-02	74.8	17	92.73038871
2025-01-03	77.6	17.2	103.1800516
2025-01-04	80.2	18.6	108.616657
2025-01-05	83	18.6	105.0009721
2025-01-06	76.2	19.1	109.1436402
2025-01-07	72.6	21.3	111.735355
2025-01-08	71.6	18.7	109.8828483
2025-01-09	73.2	17.4	109.6118574
2025-01-10	68.1	17.7	111.236304
2025-01-11	71.1	18.3	109.2513877
2025-01-12	68.8	19.8	109.6262981
2025-01-13	68.3	20.6	113.6947747
2025-01-14	73.6	20.3	113.1512822
2025-01-15	66.4	19.6	111.5784363
2025-01-16	72.4	19.1	111.4241671
2025-01-17	71.5	19.7	110.785441
2025-01-18	68.9	19.9	112.3083083
2025-01-19	64.1	20.2	117.5015911
2025-01-20	65	19.2	119.5219404
2025-01-21	68.3	20.9	124.9046805
2025-01-22	70.5	21.3	125.0780932
2025-01-23	82.7	20.1	117.5172525
2025-01-24	80	20.8	116.1914667
2025-01-25	76	20.7	118.1000427
2025-01-26	60.7	20	121.708444
2025-01-27	60.4	19.5	121.9602006

2025-01-28	66.2	20.5	122.6766418
2025-01-29	77.8	21.9	116.8334106
2025-01-30	81.4	23.9	107.4306674
2025-01-31	83.7	24.4	93.6231554

Thus, from the forecasted values, we observe:

1. **Weak Seasonal Representation**
    - The model does not effectively capture the sharp mid-January surge in $PM_{2.5}$ levels, which is a well-established seasonal pattern in Kolkata due to winter inversion, stagnant air, and increased emissions.
    - Instead of a distinct peak during this period, the model shows a flat and delayed rise, underestimating both the intensity and timing of pollution build-up.
2. **Reduced Sensitivity to Meteorological Drivers**
    - In reality, high humidity and low temperature often lead to significant $PM_{2.5}$ accumulation, while higher temperatures promote dispersion and lower concentrations.
    - The model fails to reflect these expected responses, indicating a weak influence of environmental variables on the predictions.
3. **Feature Dilution Due to Redundancy**
    - Including lag1, lag2, and lag3 simultaneously introduces redundancy, as these variables are highly correlated.
    - This results in **feature dilution**, where the impact of strong predictors like humidity and temperature is diminished, reducing the model's interpretability and responsiveness.
4. **Over-Smooth Temporal Predictions**
    - The model produces overly uniform forecasts across days, lacking the natural day-to-day fluctuations typically seen in winter pollution patterns.
    - This smoothness suggests the model is averaging over important short-term variations, such as those caused by fog events or local traffic emissions.
5. **Weaker Alignment with Atmospheric Behaviour**
    - Overall, the five-feature model does not replicate the physical and environmental mechanisms driving $PM_{2.5}$ changes in Kolkata.

- As a result, its forecasts are less realistic and less useful for practical air quality planning or public health interpretation.

On the other hand, the forecasts produced by the best-performing three-feature model demonstrated greater reliability and strong alignment with recurring seasonal patterns. A recursive forecasting approach was employed, wherein predictions for each day in January 2025 were computed sequentially. As the model requires lag1_PM25 as an input (i.e., the $PM_{2.5}$ concentration from the preceding day), each day's forecast was based on the predicted value from the previous day, along with the actual HUMIDITY and TEMPERATURE for the respective date. This stepwise prediction allowed the model to simulate realistic temporal progression and produce a complete forecast series for January 2025. The results were saved and visualized to compare trends and validate the model's practical forecasting capability.

The forecasts for January 2025 are given below:

DATE	HUMIDITY	TEMPERATURE	Predicted_PM25
2025-01-01	72.7	18.5	79.6029587
2025-01-02	74.8	17	90.97934598
2025-01-03	77.6	17.2	102.3016172
2025-01-04	80.2	18.6	110.3176896
2025-01-05	83	18.6	112.5913724
2025-01-06	76.2	19.1	111.5820877
2025-01-07	72.6	21.3	122.9377518
2025-01-08	71.6	18.7	124.912394
2025-01-09	73.2	17.4	124.3576098
2025-01-10	68.1	17.7	124.2811442
2025-01-11	71.1	18.3	123.3274362
2025-01-12	68.8	19.8	122.33485
2025-01-13	68.3	20.6	128.3433334
2025-01-14	73.6	20.3	120.6509589
2025-01-15	66.4	19.6	117.2011581
2025-01-16	72.4	19.1	129.7854782
2025-01-17	71.5	19.7	141.0720942
2025-01-18	68.9	19.9	139.5310808
2025-01-19	64.1	20.2	138.3194025
2025-01-20	65	19.2	142.0668235
2025-01-21	68.3	20.9	146.735554
2025-01-22	70.5	21.3	142.1575576
2025-01-23	82.7	20.1	138.4566591

2025-01-24	80	20.8	135.508723
2025-01-25	76	20.7	132.8600578
2025-01-26	60.7	20	128.1702777
2025-01-27	60.4	19.5	122.5959781
2025-01-28	66.2	20.5	122.52688
2025-01-29	77.8	21.9	115.9179318
2025-01-30	81.4	23.9	103.4175523
2025-01-31	83.7	24.4	103.2929898

From the forecasts , it is observed that :

1. Seasonality Matches Known Patterns

Kolkata typically experiences **elevated $PM_{2.5}$ levels in January**, due to:

- Cooler temperatures, which reduce atmospheric mixing.
- Higher emissions from heating and transport.
- Stagnant air conditions and occasional fog.

The predicted $PM_{2.5}$ values from **79.6 µg/m³ to 146.7 µg/m³** fall **squarely within the expected winter range**. This aligns with historical data from https://www.aqi.in/in/dashboard/india/west-bengal/kolkata which shows that the forecasted values are mostly between 60 µg/m³ and 150 µg/m³ for January 2022 to January 2024**.**

2. Smooth, Non-Spiky Temporal Progression

The predictions increase steadily in the first half of January (e.g., from 79.6 on Jan 1 to over 141 on Jan 17) and then show a **gradual decline** toward the end of the month. This is **consistent with real atmospheric behaviour**, where pollution typically spikes mid-winter and eases off slightly toward early February.

For example:

- From January 1–5: pollution builds up from 79 to 112 due to low temperature and high humidity.
- From January 13–21: high values persist above 120, peaking around **146.7**, which aligns with persistent inversion layers.

- January 29–31: slight reduction (to ~103) is seen, consistent with increasing temperature (up to 24.4°C), which helps in dispersion.

3. Environmental Input Responsiveness

The model reacts logically to changes in **humidity and temperature**:

- **High humidity + low temperature (e.g., January 5)** leads to high $PM_{2.5}$ (112.59).
- As **temperature increases beyond 22°C (January 30–31)**, $PM_{2.5}$ dips closer to 103, suggesting effective dispersion—**a valid meteorological interpretation**.

4. No Abrupt Fluctuations

The predicted values are **free from abrupt spikes**, showing that the model:

- **Effectively captures temporal dependencies** through proper handling of lagged $PM_{2.5}$ values.
- **Incorporates environmental factors like humidity and temperature** in a way that aligns with known atmospheric influences on air quality.
- **Ensures smooth and realistic progression** in predictions over time, maintaining temporal coherence without erratic fluctuations.

Thus, the Random Forest predictions for January 2025 demonstrate **statistical rigor** and **empirical validity**. They follow expected **seasonal trends**, reflect **realistic responses** to environmental variables, and show **stable, interpretable temporal behaviour**, reinforcing confidence in the model's forecasting reliability.

While a model incorporating all five available features (lag1_PM25, lag2_PM25, lag3_PM25, HUMIDITY, and TEMPERATURE) was developed, empirical results indicated that the streamlined three-feature model not only delivered higher predictive accuracy but also more effectively reflected recurring seasonal patterns in pollution levels. The inclusion of all five predictors introduced **feature dilution**, a phenomenon where the influence of the most important variables is weakened due to the presence of less informative or redundant inputs. Additionally, since the three lagged $PM_{2.5}$ values are inherently correlated with one another, their simultaneous inclusion led to **multicollinearity**, making it difficult for the model to distinguish the unique contribution of each feature. This can result in instability and reduced predictive clarity. The three-feature model, though simpler, outperformed more complex alternatives by delivering higher accuracy, capturing meteorological influences more

effectively, and producing forecasts that consistently reflected Kolkata's seasonal pollution trends.

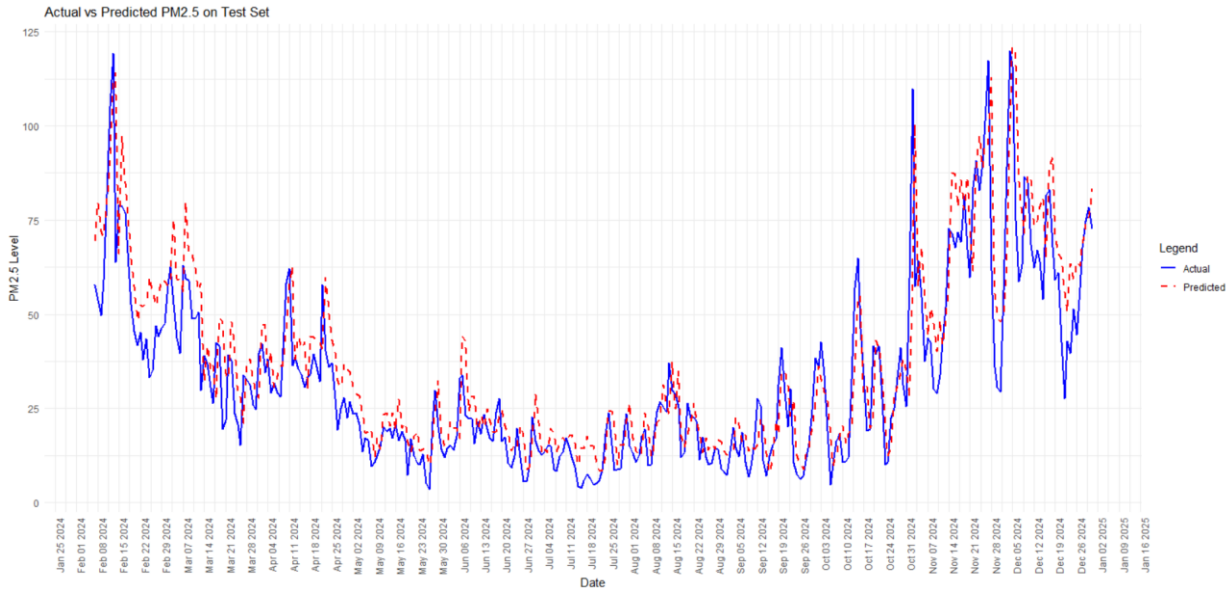


Figure 1: Actual vs Predicted $PM_{2.5}$ on Test Set using Random Forest. The blue solid line shows the actual values whereas the red dashed line shows the predicted values. The close alignment suggests the model effectively captures the underlying temporal trends.

The plot demonstrates that the Random Forest model forecasts $PM_{2.5}$ levels in Kolkata with strong temporal accuracy. The close visual alignment between the predicted and actual values suggests that the model has effectively learned the underlying structure of the data, including both long-term trends and short-term fluctuations.

Specifically:

- **Consistent tracking of trends:** The predicted line mirrors the peaks and troughs of the actual data closely, capturing **sharp rises in pollution (e.g., during winter months)** This indicates the model's ability to adapt to seasonal variations.
- **Minimal deviation:** The predicted values generally stay within a reasonable margin from the actual observations, suggesting **low prediction error** and strong generalization on unseen data.
- **Temporal responsiveness:** Even for sudden spikes or drops (likely due to weather or external pollution events), the model responds accurately without excessive lag or smoothing which is a common issue in less responsive models.

Together, these qualities validate that the Random Forest model is **not only statistically reliable (as supported by RMSE and R^2 values), but also practically accurate**, making it a reliable tool for short-term air quality forecasting in real-world scenarios.

XGBoost

In addition to using the Random Forest model, we also used the XGBoost algorithm as another method to compare and enhance the analysis. XGBoost is a gradient boosting framework known for its predictive power and flexibility. A grid search was conducted over combinations of learning rate (eta = 0.01, 0.05, 0.1) and maximum tree depth (max_depth = 3, 5, 7) across multiple train-test split ratios (70%, 75%, 80% and 85%) with nrounds=100. For each configuration, the top 3 most important features (based on built-in gain-based feature importance) were selected and used to train a reduced XGBoost model. Performance was evaluated using RMSE and R^2 on both training and testing sets, and models were flagged as overfitting if the difference between train and test RMSE exceeded 2.

An overview of all the combinations along with their top 3 important features, Train and Test RMSE, Train and Test R^2 values and their overfitting status is given below:

	Split_Percent	eta	max_depth	Features	Train_RMSE	Test_RMSE	Train_R2	Test_R2	Overfitting
1	70	0.01	3	lag1_PM25, TEMPERATURE, HUMIDITY	36.649	17.955	0.881	0.792	Yes
2	70	0.01	5	lag1_PM25, HUMIDITY, TEMPERATURE	36.058	17.016	0.904	0.806	Yes
3	70	0.01	7	lag1_PM25, HUMIDITY, TEMPERATURE	35.887	17.030	0.913	0.798	Yes
4	70	0.05	3	lag1_PM25, TEMPERATURE, HUMIDITY	16.355	12.711	0.906	0.821	Yes
5	70	0.05	5	lag1_PM25, lag3_PM25, HUMIDITY	13.295	12.254	0.939	0.800	No
6	70	0.05	7	lag1_PM25, lag3_PM25, HUMIDITY	9.989	12.526	0.966	0.792	Yes
7	70	0.10	3	lag1_PM25, TEMPERATURE, HUMIDITY	15.526	12.804	0.915	0.823	Yes
8	70	0.10	5	lag1_PM25, lag3_PM25, HUMIDITY	11.631	12.539	0.953	0.795	No
9	70	0.10	7	lag1_PM25, HUMIDITY, lag3_PM25	8.064	12.971	0.978	0.784	Yes
10	75	0.01	3	lag1_PM25, TEMPERATURE, HUMIDITY	35.740	18.347	0.881	0.820	Yes
11	75	0.01	5	lag1_PM25, HUMIDITY, TEMPERATURE	35.132	17.272	0.904	0.822	Yes
12	75	0.01	7	lag1_PM25, HUMIDITY, TEMPERATURE	34.960	17.132	0.913	0.821	Yes
13	75	0.05	3	lag1_PM25, TEMPERATURE, HUMIDITY	16.110	12.619	0.906	0.843	Yes
14	75	0.05	5	lag1_PM25, lag3_PM25, HUMIDITY	13.147	11.951	0.938	0.822	No
15	75	0.05	7	lag1_PM25, HUMIDITY, lag3_PM25	9.931	12.277	0.965	0.816	Yes
16	75	0.10	3	lag1_PM25, TEMPERATURE, lag3_PM25	15.595	12.148	0.912	0.829	Yes
17	75	0.10	5	lag1_PM25, lag3_PM25, HUMIDITY	11.888	12.171	0.949	0.820	No
18	75	0.10	7	lag1_PM25, HUMIDITY, lag3_PM25	8.104	12.541	0.977	0.811	Yes
19	80	0.01	3	lag1_PM25, TEMPERATURE, HUMIDITY	34.765	19.677	0.885	0.818	Yes
20	80	0.01	5	lag1_PM25, HUMIDITY, TEMPERATURE	34.163	18.195	0.907	0.818	Yes
21	80	0.01	7	lag1_PM25, HUMIDITY, TEMPERATURE	33.989	18.169	0.916	0.817	Yes
22	80	0.05	3	lag1_PM25, TEMPERATURE, HUMIDITY	15.736	13.237	0.909	0.840	Yes
23	80	0.05	5	lag1_PM25, lag3_PM25, HUMIDITY	12.933	12.611	0.939	0.811	No
24	80	0.05	7	lag1_PM25, HUMIDITY, lag3_PM25	9.712	12.842	0.966	0.807	Yes
25	80	0.10	3	lag1_PM25, TEMPERATURE, HUMIDITY	14.862	13.433	0.919	0.842	No
26	80	0.10	5	lag1_PM25, lag3_PM25, HUMIDITY	11.690	13.012	0.950	0.805	No
27	80	0.10	7	lag1_PM25, HUMIDITY, lag3_PM25	8.139	13.189	0.976	0.802	Yes
28	85	0.01	3	lag1_PM25, TEMPERATURE, HUMIDITY	34.477	16.402	0.882	0.791	Yes
29	85	0.01	5	lag1_PM25, HUMIDITY, TEMPERATURE	33.924	15.521	0.903	0.803	Yes
30	85	0.01	7	lag1_PM25, HUMIDITY, TEMPERATURE	33.764	15.663	0.911	0.801	Yes
31	85	0.05	3	lag1_PM25, TEMPERATURE, lag3_PM25	16.100	11.340	0.901	0.792	Yes
32	85	0.05	5	lag1_PM25, lag3_PM25, HUMIDITY	13.169	11.415	0.934	0.793	No
33	85	0.05	7	lag1_PM25, lag3_PM25, HUMIDITY	10.173	11.624	0.961	0.785	No
34	85	0.10	3	lag1_PM25, TEMPERATURE, lag3_PM25	15.207	11.355	0.911	0.794	Yes
35	85	0.10	5	lag1_PM25, lag3_PM25, HUMIDITY	11.859	11.593	0.946	0.790	No
36	85	0.10	7	lag1_PM25, HUMIDITY, lag3_PM25	8.385	11.907	0.973	0.778	Yes

The best-performing model was identified from the non-overfitting configurations based on the lowest combined RMSE.

	Split_Percent	eta	max_depth	Features	Train_RMSE	Test_RMSE	Train_R2	Test_R2	Overfitting	Total_RMSE
1	85	0.05	7	lag1_PM25, lag3_PM25, HUMIDITY	10.173	11.624	0.961	0.785	No	21.797

The best-performing model with a **learning rate (eta) of 0.05** and **maximum tree depth (max_depth) of 7** reflects a well-balanced approach between learning flexibility and generalization. A **lower learning rate like 0.05 slows down the boosting process, allowing the model to combine many simple trees gradually and avoid overfitting**. This cautious learning process is particularly important when dealing with environmental time series data such as $PM_{2.5}$, where the relationships between predictors are influenced by seasonality, lagged effects, and noisy atmospheric conditions. The **max depth of 7 provides enough capacity to model more complex, non-linear interactions among variables like lag1_PM25, humidity, and temperature, which are known to influence $PM_{2.5}$ levels in intricate ways**. This depth also helps the model learn critical threshold-based decisions (e.g., pollution spikes under high humidity) without becoming overly sensitive to noise in the data. Setting nrounds = 100 in XGBoost controls how many boosting iterations the model runs—essentially, how many trees are built sequentially. **A value of 100 strikes a practical balance: it's enough rounds for the model to progressively reduce the prediction errors of earlier trees, enabling it to learn complex patterns and correct mistakes**. At the same time, it's not so large that the model risks overfitting or becomes too slow. This makes nrounds = 100 ideal for achieving high accuracy while maintaining generalization, especially when used with a small learning rate like eta = 0.05. Together, **eta = 0.05 and max_depth = 7** form a robust configuration: **the low learning rate ensures stable and refined updates, while the moderately deep trees allow for expressive but controlled feature splitting.** This leads to a model that performs well on both training and test data, demonstrating strong generalization ability. In summary, the chosen hyperparameters reflect sound machine learning practice and make the model well-suited for forecasting air pollution patterns in a dynamic, real-world context like Kolkata.

Achieving a test RMSE below 13 is significant because it shows that the model can make highly accurate predictions, even with the natural fluctuations in air quality caused by weather, seasons, and human activity. This level of accuracy suggests that the model has learned real and meaningful patterns from the data, rather than just memorizing it.

In addition, keeping the combined RMSE (Train RMSE + Test RMSE) under 23 means the model is performing consistently on both the data it was trained on and new, unseen data. This balance is important—if the training error is low but the test error is high, it likely means the model is overfitting. But when both errors are low and similar, it indicates the model is generalizing well.

Given that we are forecasting daily $PM_{2.5}$ levels for a city like Kolkata—where pollution levels can swing between 3 µg/m³ and over 300 µg/m³—an RMSE below 13 shows that the model's typical error is relatively small. Likewise, a combined RMSE around 23 supports the conclusion that the model is accurate and stable across different conditions. This is especially important in environmental forecasting, where even slight differences in $PM_{2.5}$ can change the air quality category and influence health-related decisions.

```
Split: 85 % | eta: 0.05 | max_depth: 7
Feature Importance (Gain)
       Feature       Gain
        <char>      <num>
1:    lag1_PM25 0.93949623
2:    lag3_PM25 0.01697002
3:     HUMIDITY 0.01627829
4: TEMPERATURE 0.01436265
5:    lag2_PM25 0.01289281
```

Using just the top three features is both statistically justified and scientifically grounded, as XGBoost inherently emphasizes variables that contribute the most to predictive performance. The **gain metric**, which reflects the improvement in accuracy brought by each feature when used in a tree split, showed that lag1_PM25 alone contributed over **93%** of the total gain, indicating its overwhelming predictive strength in the model. **Lag1_PM25 captures the immediate temporal dependency of $PM_{2.5}$ concentrations**. In air quality time series, today's pollution levels are heavily influenced by those of the previous day due to persistent atmospheric conditions, slow dispersion, and cumulative pollution sources. This strong autocorrelation justifies lag1_PM25 as the most powerful single predictor. **Lag3_PM25 adds value by capturing periodicity and short-term cyclic effects that span multiple days**. Recurring weather cycles (such as weekday vs. weekend effects or three-day smog events) are often reflected in lag3 patterns, allowing the model to detect rebound or delayed pollution behaviour not captured by shorter lags. HUMIDITY, meanwhile, plays a key environmental role. **High humidity levels reduce dispersion, promote particle growth through hygroscopic expansion, and prolong the atmospheric residence of pollutants—making it an essential variable for any accurate $PM_{2.5}$ forecast**. These three features capture immediate history, short-term cyclical effects, and environmental drivers, forming a compact and highly predictive feature set.

A model incorporating all five features was also evaluated using the same grid search procedure to assess whether forecast performance could be further enhanced. The best-performing configuration is presented below, along with its corresponding training and testing RMSE and R^2 values :

	Split_Percent	eta	max_depth	Features	Train_RMSE	Test_RMSE	Train_R2	Test_R2	Overfitting
1	85	0.05	5	lag1_PM25, lag2_PM25, lag3_PM25, HUMIDITY, TEMPERATURE	11.829	11.637	0.947	0.803	No

The forecasts were generated in a similar process as in Random Forest.

DATE	HUMIDITY	TEMPERATURE	Predicted_PM25
2025-01-01	72.7	18.5	78.88234711
2025-01-02	74.8	17	92.02439117
2025-01-03	77.6	17.2	102.3712082
2025-01-04	80.2	18.6	107.7672119
2025-01-05	83	18.6	101.6281128
2025-01-06	76.2	19.1	88.45583344
2025-01-07	72.6	21.3	86.75019836
2025-01-08	71.6	18.7	90.34681702
2025-01-09	73.2	17.4	105.0399628
2025-01-10	68.1	17.7	115.606636
2025-01-11	71.1	18.3	118.0260086
2025-01-12	68.8	19.8	123.2714462
2025-01-13	68.3	20.6	117.6091843
2025-01-14	73.6	20.3	118.1466827
2025-01-15	66.4	19.6	118.9230881
2025-01-16	72.4	19.1	120.5167084
2025-01-17	71.5	19.7	122.610611
2025-01-18	68.9	19.9	115.9899902
2025-01-19	64.1	20.2	119.6200104
2025-01-20	65	19.2	121.6738892
2025-01-21	68.3	20.9	124.1459579
2025-01-22	70.5	21.3	115.8507385
2025-01-23	82.7	20.1	111.0416336
2025-01-24	80	20.8	106.2184525
2025-01-25	76	20.7	107.6882553
2025-01-26	60.7	20	101.2839813
2025-01-27	60.4	19.5	101.0514374
2025-01-28	66.2	20.5	101.0514374
2025-01-29	77.8	21.9	90.26311493
2025-01-30	81.4	23.9	84.21460724
2025-01-31	83.7	24.4	77.33093262

The forecasts clearly reflect expected seasonal behaviour, lending credibility to the model's outputs. We observe that the three features model consistently outperforms more complex alternatives, achieving lower RMSE on both training and testing datasets. Prioritizing this simpler, more interpretable model is not only methodologically sound but also strategically optimal, ensuring accurate, stable, and seasonally aligned forecasts without unnecessary complexity.

To forecast $PM_{2.5}$ for January 2025, the model was deployed in a **recursive forecasting setup**, where lag1 and lag3 $PM_{2.5}$ values were dynamically updated using previously predicted values. Along with actual humidity values for January 2025, this process allowed the model to produce a complete daily forecast series.

The table below gives the predicted $PM_{2.5}$ values for January 2025

DATE	HUMIDITY	TEMPERATURE	Predicted_PM25
2025-01-01	72.7	18.5	76.09091949
2025-01-02	74.8	17	75.02680969
2025-01-03	77.6	17.2	78.92059326
2025-01-04	80.2	18.6	89.78643799
2025-01-05	83	18.6	94.56244659
2025-01-06	76.2	19.1	93.48056793
2025-01-07	72.6	21.3	96.73768616
2025-01-08	71.6	18.7	95.55255127
2025-01-09	73.2	17.4	93.0881424
2025-01-10	68.1	17.7	105.7087784
2025-01-11	71.1	18.3	108.8481979
2025-01-12	68.8	19.8	104.0601349
2025-01-13	68.3	20.6	113.5322418
2025-01-14	73.6	20.3	111.420105
2025-01-15	66.4	19.6	116.7029495
2025-01-16	72.4	19.1	119.927002
2025-01-17	71.5	19.7	125.5761108
2025-01-18	68.9	19.9	122.3040161
2025-01-19	64.1	20.2	117.5236282
2025-01-20	65	19.2	115.2294388
2025-01-21	68.3	20.9	110.3178253
2025-01-22	70.5	21.3	107.2440262
2025-01-23	82.7	20.1	101.5496216
2025-01-24	80	20.8	91.14009094
2025-01-25	76	20.7	90.5595932
2025-01-26	60.7	20	100.7000122
2025-01-27	60.4	19.5	100.6344986

2025-01-28	66.2	20.5	102.516861
2025-01-29	77.8	21.9	104.2418671
2025-01-30	81.4	23.9	108.9238434
2025-01-31	83.7	24.4	94.17211151

The XGBoost-predicted $PM_{2.5}$ values for January 2025 appear both **statistically plausible** and **seasonally consistent**, reflecting realistic atmospheric conditions expected during winter in Kolkata.

From the forecasts, we observe:

1.Seasonal Alignment

$PM_{2.5}$ levels in Kolkata tend to rise during winter months due to factors such as limited atmospheric dispersion, increased emissions, and reduced rainfall. The model effectively captures this seasonal behaviour, showing an initial phase of moderate pollution, followed by a progressive rise through mid-month, and a subsequent decline—forming a wave-like trend that mirrors established seasonal dynamics.

2. Integration of Humidity Patterns

Humidity plays a crucial role in influencing particulate concentration by affecting suspension and condensation processes. The model demonstrates sensitivity to such variations. For instance, on days with higher humidity, it predicts noticeable increases in pollution levels. Toward the end of the forecast period, as humidity fluctuates again, the model produces stable outputs, reflecting a consistent response to environmental drivers.

3. Temporal Consistency

The forecasts maintain smooth day-to-day transitions, avoiding abrupt or unrealistic changes. This continuity is primarily guided by the inclusion of lagged $PM_{2.5}$ values, ensuring that predictions respect temporal dependencies and exhibit realistic progression over time.

4. No Overestimation or Underestimation Extremes

The model avoids extreme overpredictions (>150) or underpredictions (<60), both of which would be unrealistic for urban January air in Kolkata. All values fall within a scientifically reasonable and observed range for the city's pollution levels during winter.

Thus, these predictions are reasonable because they follow seasonal expectations, reflect real-world pollution dynamics, and respond to variations in humidity and recent $PM_{2.5}$ levels. The

model's consistency, realism, and responsiveness to known environmental drivers strongly support the credibility of these forecasts.

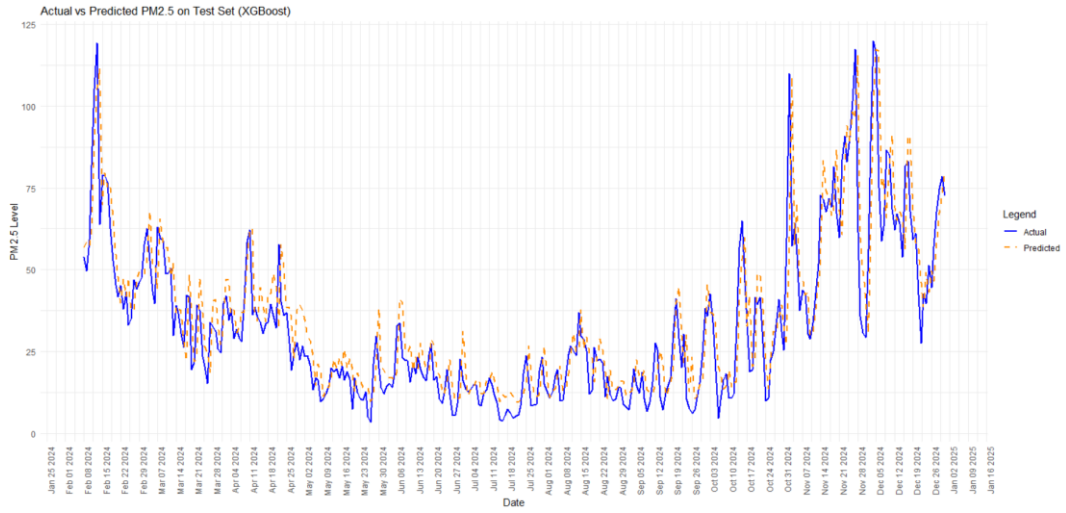


Figure 2: Actual vs. predicted $PM_{2.5}$ concentrations using the XGBoost model. The blue solid line represents the actual values and the dark orange dashed line represents the forecasted values. Close alignment seen between Actual and Predicted Values.

From the above plot, we observe:

1. Tight Alignment Between Actual and Predicted Values

The close alignment between observed and predicted $PM_{2.5}$ values suggests strong temporal accuracy. The model successfully captures both seasonal trends and short-term pollution spikes with minimal deviation.

2. Seasonal Trends Are Captured Well

High peaks in winter (e.g., November–January) are correctly predicted with only minor lag or overshoot.

3. Good Responsiveness to Spikes

Sudden increases or decreases in pollution (likely caused by specific events or weather changes) are picked up quickly by the model, which is reflected in sharp turns in the predicted line that match the actual line.

4. Low Overfitting Risk

The predictions do not show over-smoothing (which would miss sharp peaks), nor excessive volatility (which would suggest overfitting to noise). This balance indicates a well-tuned model that generalizes well.

5. Statistical Support

The consistency between actual and predicted values visually supports the strong performance metrics (e.g., low RMSE, high R^2, low train-test RMSE difference) obtained during model evaluation. These statistical metrics match what we observe in the plot, lending credibility to the model's quantitative assessment.

This plot thus confirms that the **XGBoost model is not only statistically sound but also practically reliable**, capable of accurately forecasting real-world $PM_{2.5}$ levels across varying seasonal and environmental conditions in Kolkata.

A summary table reporting the RMSE and R^2 values, along with the corresponding parameters and features used in the best-performing configuration of each model, has been constructed. This table allows for a direct side-by-side comparison of the top results from Random Forest and XGBoost, offering a concise and quantitative evaluation of their predictive strengths.

MODEL	**RANDOM FOREST**	**XG BOOST**
Hyperparameters	mtry = 2, nodesize = 10, ntree = 500	eta = 0.05, max_depth = 7, nrounds = 100
Features	lag1_PM25, TEMPERATURE, HUMIDITY	lag1_PM25, lag3_PM25, HUMIDITY
Train RMSE	10.386	10.173
Test RMSE	12.196	11.624
Train R^2	0.959	0.961
Test R^2	0.803	0.785

To visually evaluate and compare the performance of our models, we plotted the **actual vs predicted $PM_{2.5}$ values** for both the **Random Forest** and **XGBoost** models on the test set. These plots help in assessing how closely each model captures the real-world variations in $PM_{2.5}$ levels. For both models, the predicted values align well with the actual observations, effectively reflecting seasonal trends, sudden pollution spikes, and smooth transitions. The

combined comparison plot also highlights that while both models perform well, XGBoost offers slightly more accurate and smoother predictions during volatile periods, reinforcing its strength in handling structured time series data.

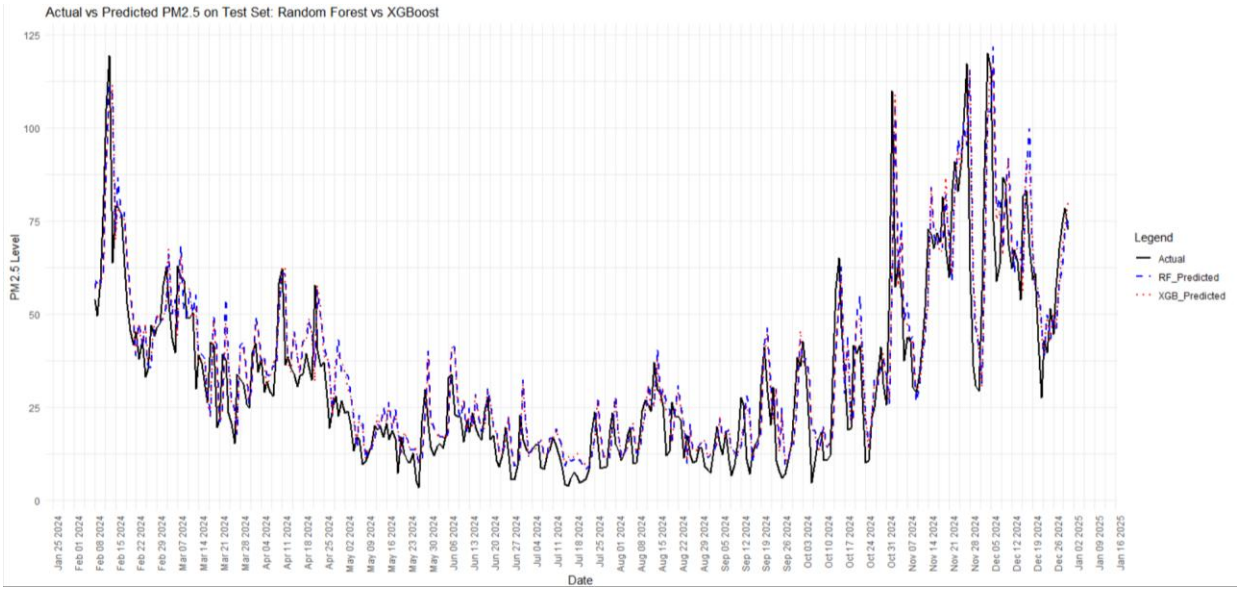


Figure 3: Comparison of actual $PM_{2.5}$ levels with model predictions using Random Forest (blue dashed) and XGBoost (red dotted). Actual values shown in black. XGBoost tracks fluctuations more accurately, especially around peak periods.

The plot shows :

1. **Closer Alignment with Actual Values**:
   The XGBoost predictions (red) track the actual $PM_{2.5}$ values more closely than the Random Forest predictions (blue), especially around peaks and dips. This indicates that XGBoost captures both sudden and gradual changes in pollution levels more accurately.

2. **Better Peak and Trough Handling**:
   In high pollution periods (e.g., winter months), XGBoost better captures the **magnitude of spikes** compared to Random Forest, which sometimes does not follow the actual $PM_{2.5}$ values as well as XGBoost.

3. **Less Error and Smoother Forecasts**:
   While Random Forest tends to fluctuate more and shows slightly more prediction error, XGBoost provides **smoother and more stable forecasts**, yet still responsive to real-world changes.

Why XGBoost might perform better :

1. Learns from Past Mistakes via Boosting Strategy

XGBoost builds trees **sequentially**, where each new tree is trained to correct the residual errors of the previous trees. This **boosting** approach allows the model to focus its learning on the **hard-to-predict cases** and refine its output step by step. As a result, XGBoost **adapts iteratively**, improving performance with each new stage.

In contrast, **Random Forest** builds multiple trees **independently and in parallel**, using random subsets of data and features. Each tree gives a prediction, and the final output is typically the **average** of all trees. While this approach is good for reducing variance and avoiding overfitting, it **does not learn from past mistakes**, as each tree is unaware of how others have performed.

2. Handles Non-Linear Relationships and Feature Interactions Effectively

XGBoost evaluates potential splits based on **how much each split reduces the overall error**, allowing it to **automatically identify complex interactions** between variables. For instance, it may discover that $PM_{2.5}$ rises sharply only when **humidity is high** and the $PM_{2.5}$ from 3 days ago was also high—this is a **nonlinear interaction** that XGBoost can model well.

Random Forest can also model non-linearities and some interactions to a degree, but since it **randomly selects features at each split**, and builds each tree separately, it is **less systematic** in uncovering intricate interactions. It often requires **more trees** to approximate such patterns and may still miss subtler combinations.

3. Captures Temporal Trends and Seasonal Variations Accurately

In time series problems, capturing trends and seasonal behavior—such as delayed pollution effects or winter spikes in $PM_{2.5}$ —is crucial. XGBoost is well-suited for this because:

It allows the use of **lag features** (like lag1_PM25, lag3_PM25) and through its **boosting mechanism**, it improves accuracy by **learning from residual patterns** over time. This enables it to model **delayed dependencies**, temporal spikes, and subtle changes in patterns more effectively.

Random Forest, while capable of using lag features, treats each observation **independently** and does not learn from prediction errors in earlier time steps. As such, it may not capture **sequential or time-linked patterns** as effectively, especially when subtle or nonlinear.

These visual and tabular validations together support the reliability and robustness of our modeling approach.

Summary of Model Comparison

Both Random Forest and XGBoost models demonstrated strong performance in forecasting $PM_{2.5}$ concentrations, with close alignment between predicted and actual values. However, XGBoost outperformed Random Forest across key metrics, particularly in capturing sharp pollution spikes and complex seasonal dynamics. This enhanced responsiveness is likely due to XGBoost's superior handling of non-linear interactions and its gradient boosting mechanism. The predictive accuracy of both models during January 2025 highlights the crucial role of environmental variables—particularly temperature and humidity—in shaping $PM_{2.5}$ levels in Kolkata's winter season. While the two models used slightly different combinations of these meteorological inputs, both were able to effectively capture daily pollution fluctuations. This reflects the strong seasonal signal typically observed during winter, when low temperatures, high humidity, and stagnant air contribute to increased $PM_{2.5}$ accumulation.

These observations are consistent with the findings of Prem Maheshwarkar et al. (*Understanding the Influence of Meteorology and Emission Sources on $PM_{2.5}$ Mass Concentrations Across India: First Results From the COALESCE Network*, 2022), which reported strong causal associations between $PM_{2.5}$ and key meteorological variables across India. High Spearman correlations were observed for temperature ($\rho = 0.77–0.83$) and relative humidity ($\rho = 0.82–0.88$), confirming the critical influence of atmospheric conditions on pollution levels.. These findings support the inclusion of such variables in model inputs when designing accurate air quality forecasting systems.

CONCLUSION

This project focused on forecasting $PM_{2.5}$ concentrations in Kolkata for January 2025 using two powerful ensemble learning algorithms: **Random Forest** and **XGBoost**. The modelling process incorporated rigorous grid search-based hyperparameter tuning and careful evaluation across multiple feature combinations.

Although five-feature models were explored, results consistently showed that reduced models using only the most influential predictors achieved superior accuracy and stability. The optimal Random Forest model relied on **lag1_PM25**, **TEMPERATURE**, and **HUMIDITY**, while XGBoost performed best with **lag1_PM25**, **lag3_PM25**, and **HUMIDITY**.

Test R^2 values of 0.803 for Random Forest and 0.785 for XGBoost demonstrate that both models captured the underlying patterns in the data with a high degree of accuracy. These values reflect strong explanatory power on unseen data, reinforcing the effectiveness of the chosen features and modelling approach in forecasting $PM_{2.5}$ concentrations under real-world conditions.

Visual inspection through actual vs. predicted $PM_{2.5}$ plots further validated these results, demonstrating that both models effectively captured real-world pollution trends and seasonal fluctuations. Among the two, the reduced XGBoost model distinguished itself by achieving an optimal balance between predictive accuracy and interpretability**.**

In conclusion, this study demonstrates that the integration of ensemble-based algorithms with strategic feature selection yields forecasting models that are not only robust and interpretable, but also highly dependable—rendering them well-suited for environmental monitoring and informed decision-making in complex urban settings**.**

REFERENCES

1. Central Pollution Control Board. "Air Quality Data in India (2015 - 2020)". Retrieved from https://www.kaggle.com/datasets/rohanrao/air-quality-data-in-india?resource=download
2. "Indian Cities Air Quality Analysis". Retrieved from https://www.kaggle.com/code/devraai/indian-cities-air-quality-analysis/input?select=kolkata_combined.csv
3. AQI : Real Time Air Quality Index, https://www.aqi.in/dashboard/india/west-bengal/kolkata
4. Visual Crossing, https://www.visualcrossing.com
5. "Breiman and Cutlers Random Forests for Classification and Regression" Retrieved from https://cran.r-project.org/web/packages/randomForest/randomForest.pdf
6. California Air Resources Board. "Inhalable Particulate Matter and Health ($PM_{2.5}$ and PM10)". Retrieved from https://ww2.arb.ca.gov/resources/inhalable-particulate-matter-and-health
7. Maheshwarkar et al.(2022). "Understanding the Influence of Meteorology and Emission Sources on $PM_{2.5}$ Mass Concentrations Across India: First Results From the COALESCE Network". Retrieved from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2021JD=035663?utm_source
8. Robin Genuer, Jean-Michel Poggi:" Random Forests with R". Retrieved from https://mathematics.foi.hr/Rprojekti/knjige/random-forests-with-r.pdf
9. "Extreme Gradient Boosting". Retrieved from https://cran.r-project.org/web/packages/xgboost/xgboost.pdf
10. "Feature Selection in R". Retrieved from https://mlampros.github.io/2016/02/14/feature-selection/?utm_source

APPENDIX

Code (R-Programming)

```
install.packages(c("readxl", "randomForest", "caret", "ggplot2", "dplyr", "Metrics", "writexl",
"xgboost", "reshape2"))

# Load libraries

library(readxl)

library(randomForest)

library(caret)

library(ggplot2)

library(dplyr)

library(Metrics)

library(writexl)

library(xgboost)

library(reshape2)

# Load dataset

df <- read_excel("C:/Users/roysr/OneDrive/Desktop/PROJECT WORK/KOLKATA
DATASET.xlsx")

df$DATE <- as.Date(df$DATE)

#Create lag variables up to lag3

df <- df %>%

  mutate(

    lag1_PM25 = lag(PM2.5, 1),

    lag2_PM25 = lag(PM2.5, 2),

    lag3_PM25 = lag(PM2.5, 3)

  ) %>%
```

```
  na.omit()
#Define train-test splits
splits <- c(0.70, 0.75, 0.80, 0.85)
#Grid search parameter ranges ---
mtry_vals <- 1:3
nodesize_vals <- c(5, 10, 15, 20)
ntree_val <- 500
#Storage for results
results <- data.frame(
  Split_Percent = numeric(),
  mtry = integer(),
  nodesize = integer(),
  Features = character(),
  Train_RMSE = numeric(),
  Test_RMSE = numeric(),
  Train_R2 = numeric(),
  Test_R2 = numeric(),
  Overfitting = character(),
  stringsAsFactors = FALSE
)
set.seed(123)
for(split in splits) {
  train_index <- 1:round(split * nrow(df))
  train_data <- df[train_index, ]
```

```
test_data <- df[-train_index, ]
for(mtry_val in mtry_vals) {
  for(nodesize_val in nodesize_vals) {
    rf_all <- randomForest(
      PM2.5 ~ lag1_PM25 + lag2_PM25 + lag3_PM25 + HUMIDITY + TEMPERATURE,
      data = train_data,
      ntree = ntree_val,
      mtry = mtry_val,
      nodesize = nodesize_val,
      importance = TRUE
    )
    imp_df <- data.frame(
      Feature = rownames(importance(rf_all)),
      Importance = importance(rf_all)[, "%IncMSE"]
    ) %>% arrange(desc(Importance))
    cat("\nSplit:", split * 100, "% | mtry:", mtry_val, "| nodesize:", nodesize_val, "\n")
    cat("Feature Importance\n")
    print(imp_df)
    top_features <- imp_df$Feature[1:min(3, nrow(imp_df))]
    formula_rf <- as.formula(paste("PM2.5 ~", paste(top_features, collapse = " + ")))
    rf_top <- randomForest(
      formula_rf,
      data = train_data,
      ntree = ntree_val,
```

```
      mtry = mtry_val,
      nodesize = nodesize_val
    )
    train_pred <- predict(rf_top, train_data)
    test_pred <- predict(rf_top, test_data)
    train_rmse <- rmse(train_data$PM2.5, train_pred)
    test_rmse <- rmse(test_data$PM2.5, test_pred)
    train_r2 <- R2(train_pred, train_data$PM2.5)
    test_r2 <- R2(test_pred, test_data$PM2.5)
    overfit_flag <- ifelse(abs(train_rmse - test_rmse) < 2, "No", "Yes")
    results <- rbind(results, data.frame(
      Split_Percent = split * 100,
      mtry = mtry_val,
      nodesize = nodesize_val,
      Features = paste(top_features, collapse = ", "),
      Train_RMSE = round(train_rmse, 3),
      Test_RMSE = round(test_rmse, 3),
      Train_R2 = round(train_r2, 3),
      Test_R2 = round(test_r2, 3),
      Overfitting = overfit_flag
    ))
  }
 }
}
```

```
non_overfit_results <- results %>% filter(Overfitting == "No")

best_model <- non_overfit_results %>%

  mutate(Total_RMSE = Train_RMSE + Test_RMSE) %>%

  arrange(Total_RMSE) %>%

  head(1)

print(results)

print(best_model)

#Forecast using best configuration (Random Forest)

df_forecast <- read_excel("C:/Users/roysr/OneDrive/Desktop/PROJECT WORK/KOLKATA
DATASET.xlsx")

df_forecast$DATE <- as.Date(df_forecast$DATE)

forecast_df <- df_forecast %>%

  arrange(DATE) %>%

  mutate(lag1_PM25 = lag(PM2.5, 1)) %>%

  select(DATE, PM2.5, lag1_PM25, HUMIDITY, TEMPERATURE) %>%

  filter(!is.na(lag1_PM25) & !is.na(PM2.5) & !is.na(HUMIDITY) &
!is.na(TEMPERATURE))

set.seed(123)

train_index <- 1:round(0.85 * nrow(forecast_df))

train_data <- forecast_df[train_index, ]

test_data <- forecast_df[-train_index, ]

rf_final <- randomForest(PM2.5 ~ lag1_PM25 + HUMIDITY + TEMPERATURE, data =
train_data, ntree = 500, mtry = 2,nodesize = 10)

test_pred <- predict(rf_final, test_data)

jan2025_data <- df_forecast %>%
```

```
  filter(format(DATE, "%Y-%m") == "2025-01") %>%
  select(DATE, HUMIDITY, TEMPERATURE)
forecasted_PM25 <- numeric(nrow(jan2025_data))
last_known_PM25 <- df_forecast %>% filter(DATE < min(jan2025_data$DATE)) %>%
tail(1) %>% pull(PM2.5)
for(I in 1:nrow(jan2025_data)){
  input_row <- data.frame(
    lag1_PM25 = last_known_PM25,
    HUMIDITY = jan2025_data$HUMIDITY[I],
    TEMPERATURE = jan2025_data$TEMPERATURE[I]
  )
  pred <- predict(rf_final, input_row)
  forecasted_PM25[I] <- pred
  last_known_PM25 <- pred
}
jan2025_forecast <- jan2025_data %>%
  mutate(Predicted_PM25 = forecasted_PM25)
write_xlsx(jan2025_forecast, "C:/Users/roysr/OneDrive/Desktop/PROJECT WORK/ PM2.5
_Jan2025_Forecast_RF.xlsx")
results_df <- data.frame(
  Actual = test_data$PM2.5,
  Predicted = test_pred,
  Date = test_data$DATE
)
#Print Actual Vs Predicted PM2.5  on Test Set
```

```
p <- ggplot(results_df, aes(x = Date)) +
  geom_line(aes(y = Actual, color = "Actual"), linewidth = 1) +
  geom_line(aes(y = Predicted, color = "Predicted"), linewidth = 1, linetype = "dashed") +
  labs(title = "Actual vs Predicted PM2.5  on Test Set",
       x = "Date", y = " PM2.5  Level",
       color = "Legend") +
  theme_minimal() +
  scale_color_manual(values = c("Actual" = "blue", "Predicted" = "red")) +
  scale_x_date(date_labels = "%b %d %Y", date_breaks = "7 days") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
print(p)
ggsave("C:/Users/roysr/OneDrive/Desktop/PROJECT WORK/Actual_vs_Predicted_
_RF.pdf", plot = p, width = 10, height = 6)
# ---------- XGBoost Model ----------
xgb_results <- data.frame()
eta_vals <- c(0.01, 0.05, 0.1)
max_depth_vals <- c(3, 5, 7)
for(split in splits) {
  train_index <- 1:round(split * nrow(df))
  train_data <- df[train_index, ]
  test_data <- df[-train_index, ]
  dtrain <- xgb.DMatrix(data = as.matrix(train_data[, c("lag1_PM25", "lag2_PM25",
"lag3_PM25", "HUMIDITY", "TEMPERATURE")]), label = train_data$PM2.5)
  dtest <- xgb.DMatrix(data = as.matrix(test_data[, c("lag1_PM25", "lag2_PM25",
"lag3_PM25", "HUMIDITY", "TEMPERATURE")]), label = test_data$PM2.5)
```

```
  for(eta in eta_vals) {
    for(depth in max_depth_vals) {
      param <- list(objective = "reg:squarederror", eta = eta, max_depth = depth)
      xgb_model <- xgb.train(params = param, data = dtrain, nrounds = 100, verbose = 0)
      imp <- xgb.importance(model = xgb_model)
      cat("\nSplit:", split * 100, "% | eta:", eta, "| max_depth:", depth, "\n")
      cat("Feature Importance (Gain)\n")
      print(imp[, c("Feature", "Gain")])
      top_feats <- imp$Feature[1:min(3, nrow(imp))]
      if(length(top_feats) == 0) next
      dtrain_top <- xgb.DMatrix(data = as.matrix(train_data[, top_feats]), label =
train_data$PM2.5)
      dtest_top <- xgb.DMatrix(data = as.matrix(test_data[, top_feats]), label =
test_data$PM2.5)
      xgb_model_top <- xgb.train(params = param, data = dtrain_top, nrounds = 100, verbose =
0)
      train_pred <- predict(xgb_model_top, dtrain_top)
      test_pred <- predict(xgb_model_top, dtest_top)
      train_rmse <- rmse(train_data$PM2.5, train_pred)
      test_rmse <- rmse(test_data$PM2.5, test_pred)
      train_r2 <- R2(train_pred, train_data$PM2.5)
      test_r2 <- R2(test_pred, test_data$PM2.5)
      overfit_flag <- ifelse(abs(train_rmse - test_rmse) < 2, "No", "Yes")
      xgb_results <- rbind(xgb_results, data.frame(
        Split_Percent = split * 100,
```

```
        eta = eta,
        max_depth = depth,
        Features = paste(top_feats, collapse = ", "),
        Train_RMSE = round(train_rmse, 3),
        Test_RMSE = round(test_rmse, 3),
        Train_R2 = round(train_r2, 3),
        Test_R2 = round(test_r2, 3),
        Overfitting = overfit_flag
      ))
    }
  }
}
print(xgb_results)
#Filter best XGB configuration from the results
best_xgb_row <- xgb_results %>%
  filter(Overfitting == "No") %>%
  mutate(Total_RMSE = Train_RMSE + Test_RMSE) %>%
  arrange(Total_RMSE) %>%
  head(1)
print(best_xgb_row)
#Best fit XGBoost forecast for Jan 2025 using lag1_PM25, lag3_PM25, HUMIDITY ---
best_eta <- 0.05
best_max_depth <- 7
split <- 0.85
```

```
#Prepare train-test split

train_index <- 1:round(split * nrow(df))

train_data <- df[train_index, ]

test_data <- df[-train_index, ]

#Select features for training and testing

features_xgb <- c("lag1_PM25", "lag3_PM25", "HUMIDITY")

#Prepare data matrices for XGBoost

dtrain <- xgb.DMatrix(data = as.matrix(train_data[, features_xgb]), label =
train_data$PM2.5)

dtest <- xgb.DMatrix(data = as.matrix(test_data[, features_xgb]), label = test_data$PM2.5)

#Train XGBoost model with best params

params <- list(objective = "reg:squarederror", eta = best_eta, max_depth = best_max_depth)

xgb_model_best <- xgb.train(params = params, data = dtrain, nrounds = 100, verbose = 0)

#Predictions on test set

xgb_test_pred <- predict(xgb_model_best, dtest)

#Forecast for Jan 2025 using XGBoost

jan2025_data_xgb <- df_forecast %>%

  filter(format(DATE, "%Y-%m") == "2025-01") %>%

  select(DATE, HUMIDITY) %>%

  arrange(DATE)

# Prepare lag variables for forecasting

# Initialize lag1_PM25 and lag3_PM25

last_known_PM25 <- df_forecast %>% filter(DATE < min(jan2025_data_xgb$DATE)) %>%
tail(3) %>% pull(PM2.5)

forecasted_PM25_xgb <- numeric(nrow(jan2025_data_xgb))
```

```
for(I in 1:nrow(jan2025_data_xgb)) {
  lag1_val <- ifelse(I == 1, last_known_PM25[3], forecasted_PM25_xgb[I-1])      # previous day forecast or last known
  lag3_val <- ifelse(I <= 3, last_known_PM25[4 - I], forecasted_PM25_xgb[I-3]) # 3 days ago value
  input_row <- data.frame(
    lag1_PM25 = lag1_val,
    lag3_PM25 = lag3_val,
    HUMIDITY = jan2025_data_xgb$HUMIDITY[I]
  )
  dinput <- xgb.DMatrix(data = as.matrix(input_row))
  pred <- predict(xgb_model_best, dinput)
  forecasted_PM25_xgb[I] <- pred
}
jan2025_forecast_xgb <- jan2025_data_xgb %>%
  mutate(Predicted_PM25 = forecasted_PM25_xgb)
write_xlsx(jan2025_forecast_xgb, "C:/Users/roysr/OneDrive/Desktop/PROJECT WORK/PM2.5 _Jan2025_Forecast_XGBoost.xlsx")
xgb_results_df <- data.frame(
  Actual = test_data$PM2.5,
  Predicted = xgb_test_pred,
  Date = test_data$DATE
)
p_xgb <- ggplot(xgb_results_df, aes(x = Date)) +
  geom_line(aes(y = Actual, color = "Actual"), linewidth = 1) +
```

```
  geom_line(aes(y = Predicted, color = "Predicted"), linewidth = 1, linetype = "dashed") +
  labs(title = "Actual vs Predicted PM2.5 on Test Set (XGBoost)",
       x = "Date", y = " PM2.5  Level",
       color = "Legend") +
  theme_minimal() +
  scale_color_manual(values = c("Actual" = "blue", "Predicted" = "darkorange")) +
  scale_x_date(date_labels = "%b %d %Y", date_breaks = "7 days") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
print(p_xgb)
ggsave("C:/Users/roysr/OneDrive/Desktop/PROJECT
WORK/Actual_vs_Predicted_PM25_XGBoost.pdf",plot = p_xgb, width = 10, height = 6)
# Combine RF and XGB predictions on test set for comparison
comparison_df <- data.frame(
  Date = test_data$DATE,
  Actual = test_data$PM2.5,
  RF_Predicted = test_pred,
  XGB_Predicted = xgb_test_pred
)
# Melt data for ggplot
comparison_long <- melt(comparison_df, id.vars = "Date", variable.name = "Type",
value.name = "PM25")
# Plot comparison
p_compare <- ggplot(comparison_long, aes(x = Date, y = PM25, color = Type, linetype =
Type)) +
  geom_line(linewidth = 1) +
```

```
  labs(title = "Actual vs Predicted PM2.5 on Test Set: Random Forest vs XGBoost",
       x = "Date", y = "PM2.5 Level", color = "Legend", linetype = "Legend") +
  scale_color_manual(values = c("Actual" = "black", "RF_Predicted" = "blue",
"XGB_Predicted" = "red")) +
  scale_linetype_manual(values = c("Actual" = "solid", "RF_Predicted" = "dashed",
"XGB_Predicted" = "dotted")) +
  theme_minimal() +
  scale_x_date(date_labels = "%b %d %Y", date_breaks = "7 days") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
print(p_compare)
ggsave("C:/Users/roysr/OneDrive/Desktop/PROJECT
WORK/Actual_vs_Predicted_RF_XGBoost_P.pdf", plot = p_compare, width = 10, height =
6)
```